

**A MINI PROJECT REPORT
ON
Health Assessment and Performance Prediction in Cloud Environments
using Ensemble Modeling**

SUBMITTED TO AN AUTONOMOUS INSTITUTE, AFFILIATED TO
SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE IN THE PARTIAL
FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF
THE DEGREE
OF

**T.Y. BACHELOR OF TECHNOLOGY
CSE (DATASCIENCE)**

SUBMITTED BY

Anupriya Bhoyar	RollNo : 10
Dhanashree Khamitkar	RollNo: 14
Ishika Singh	RollNo: 21



**DEPARTMENT OF CSE (DATA SCIENCE) G H RAISONI COLLEGE
OF ENGINEERING AND MANAGEMENT
WAGHOLI, PUNE 412207**

2025-26



G H Raison
COLLEGE

Engineering and
Management
Pune

An Empowered Autonomous Institute, Affiliated to SPPU, Pune
NAAC Accredited "A+" Grade & NBA

CERTIFICATE

This is to certify that the Mini Project report entitles

**Health Assessment and Performance Prediction in Cloud Environments
using Ensemble Modeling**

Submitted by

Anupriya Bhoyar	Roll No : 10
Dhanashree Khamitkar	Roll No: 14
Ishika Singh	Roll No: 21

are a bonafide students of this institute and the work has been carried out by them under the supervision of Prof. Chiranjit Das and it is approved for the partial fulfillment of the requirement of An Autonomous Institute, Affiliated to Savitribai Phule Pune University, for the award of the degree of **T.Y. Bachelor of Technology in CSE[DATA SCIENCE]** in the academic year 2025-26.

(Prof. Chiranjit Das)
Guide

(Prof. Rahul Borate)
MiniprojectCoordinator

(Dr. Deepika Ajalkar)
H.O.D.

(Dr. N. B. Hulle)
I/C Director
GHRCEM, Pune

ACKNOWLEDGMENT

It gives us great pleasure in presenting **Health Assessment and Performance Prediction in Cloud Environments using Ensemble Modeling** as our T.Y. B.Tech. Mini project. Words have never seemed as inadequate as now when we are endeavoring to express our gratitude at the culmination of our T.Y. B.Tech. Mini project to all those who have made it possible. Even the best efforts are waste, without the proper guidance and advice of our project guide **Prof. Chiranjit Das** for the consistent guidance, co-operation, inspiration, practical approach and constructive criticism, which provided us the much needed impetus to work hard & also thanks **Dr. Deepika Ajalkar** Head of CSE (Data Science) Department for their continuous support & valuable suggestions. We take this opportunity to thank our I/C Director **Dr. N. B. Hulle** for their whole hearted support, motivation & valuable suggestions. We would also like to thank **Prof. Rahul Borate** our Mini project Coordinator, for her valuable support in providing us with the required information. At the end, we would like to give special thanks to all staff members from CSE(**Data Science**) of G H Raison College of Engineering and Management, Pune & our colleagues for their kind support & timely suggestions.

Anupriya Bhoyar

Roll No : 10

Dhanashree Khamitkar

Roll No : 14

Ishika Singh

Roll No : 21

ABSTRACT

A cloud environment is a dynamic platform that provides the required resources, storage, services and many more over the internet. The rapid increase in usage of cloud environments for its efficiency, scalability and fast computation, can also come with the performance degradation and failures. Here health assessment plays a crucial role by predicting the failures in advance. The existing systems are using single machine learning models and are able to detect but after the failures has already occurred and they lack the full architecture of classifying systems in different states. They are heavily depended on the historical data storage. To address these all limitations this project proposes a stateless cloud monitoring system that will combine the performance prediction and health assessment using ensemble machine learning models. It enables the early detection of all possible failures. Multiple machine learning models will be used together to predict the performance behavior and classify the system as Normal, Warning, or Critical accordingly. The system is going to depend on real time rather than the stored one. It makes this more reliable because of more accuracy, scalability and suitable for the modern cloud applications. The solution aims to provide a cloud system as a smooth and safe environment for the users. By checking this in advance it will prevent wastage of resources and increases the reliability and efficiency in cloud environments.

Keywords: Cloud Computing , Health Assessment , Ensemble Machine Learning , Predictive Analytics

TABLE OF CONTENTS

CERTIFICATE	Page No. i
ACKNOWLEDGMENT	ii
ABSTRACT	iii
TABLE OF CONTENT	iv
LIST OF FIGURES	v
LIST OF TABLES	vi

Sr. No.	Title	Page No.
1	Abstract	3
2	Acknowledgement	2
3	List of Figures	6
4	List of Tables	
5	Chapter 1: Introduction	7
	1.1 Background of the Study	7
	1.2 Objectives	7
	1.3 Problem Statement	7
	1.4 Scope of the Project	8
6	Chapter 2: Literature Review	8
	2.1 Existing Systems	8
	2.2 Summary of Reviewed Literature	8
7	Chapter 3: Methodology	10
	3.1 Working Methodology	10
	3.2 System Architecture	11
	3.3 System Requirements	12
	3.3.1 Hardware Requirements	12
	3.3.2 Software Requirements	12

	3.4 Tools and Technologies Used	13
8	Chapter 4: System Design	13
	4.1 Block Diagram	14
	4.2 Use Case Diagram	16
	4.3 Data Flow Diagram (DFD)	16
	4.3.1 Level 0 DFD	16
	4.3.2 Level 1 DFD	18
	4.4 Class Diagram	19
9	Chapter 5: Implementation	21
	5.1 Module Description	21
	5.2 Algorithm / Working Logic	23
	5.3 System Implementation (Code Snippets)	23
	5.4 Output Screenshots	26
10	Chapter 6: Testing and Results	28
	6.1 Test Plan	28
	6.2 Test Cases	28
	6.3 Analysis of Results	29
	6.4 Output Verification Screenshots	30
11	Chapter 7: System Evaluation and Future Scope	32
	7.1 Advantages	32
	7.2 Limitations	32
	7.3 Future Enhancements	33
	7.4 Applications	33
12	Chapter 8: Conclusion	33
13	Chapter 9: References	35

LIST OF FIGURES

FIGURE NO	ILLUSTRATION	PAGE NO
3.2.1	System Architecture	11
4.1.1	Block Diagram	14
4.2.1	Use Case Diagram	15
4.3.1	Data Flow Diagram lvl 0	16
4.3.2	Data Flow Diagram lvl 1	18
4.4.1	Class Diagram	20
5.4.1	Frontend Dashboard	26
5.4.2	FastAPI	27
5.4.3	Terminal Logs	27
6.4.1	Response	30

LIST OF TABLES

TABLE	ILLUSTRATION	PAGE NO.
1	Literature Review	9
2	Test Cases	28

Chapter 1: Introduction

1.1 Background

Cloud computing provides on-demand access to resources like servers, storage, and applications over the internet on a pay-as-you-go basis, enabling services such as email and streaming without managing hardware. Monitoring ensures performance, availability, and security by detecting issues, reducing downtime, and optimizing costs. However, it faces challenges like data security risks, downtime, vendor lock-in, and compliance issues. Machine learning improves cloud systems by predicting workloads, optimizing resources, and detecting failures, making them more efficient and reliable.

1.2 Problem Statement

Most cloud monitoring systems detect problems only after they happen and depend on stored past data. This makes them slow, complex, and difficult to scale. There is a need for a simple, stateless system that can predict cloud performance in real time and show the overall system health early using machine learning.

1.3 Objectives

The proposed system is developed to collect raw cloud resource data such as CPU usage, memory consumption, disk utilization, and network activity provided by users, which serves as the input for further analysis and processing. Using this data, the system aims to predict cloud system performance by applying ensemble machine learning models that can estimate future performance behavior more accurately and reliably than a single model. This prediction helps in understanding how the system may perform under different conditions and workloads. In addition to performance prediction, the system also classifies the health status of the cloud system into simple categories such as Normal, Warning, or Critical, allowing users to quickly interpret the condition of their system and take necessary actions. Furthermore, the entire solution is designed as a full-stack stateless system that combines frontend, backend, and machine learning components into a scalable architecture, while avoiding the storage of past user data to maintain simplicity, efficiency, and flexibility.

1.4 Scope of the Project

The scope of this project is to develop a machine learning-based system that can monitor and predict cloud performance in real time without relying heavily on historical data. The system focuses on building a simple and scalable stateless model that provides early insights into system health, helping in proactive issue detection. It aims to improve performance monitoring, reduce downtime, and enhance resource utilization. Additionally, the project can be extended to detect anomalies, predict failures, and support efficient decision-making in cloud environments.

Chapter 2: Literature Review

2.1 Introduction to Existing Systems

The development of cloud monitoring and predictive health assessment systems has gained significant attention in recent years due to the increasing complexity and scale of cloud computing environments. Traditional monitoring systems primarily rely on single machine learning models to observe system behavior and detect anomalies. While these approaches provide basic insights, they often face several limitations, including a reactive nature where issues are identified only after they occur, a heavy dependency on large volumes of historical data, and a lack of ability to classify system states into clear, actionable categories for decision-making. As cloud infrastructures continue to grow in size and dynamic behavior, these limitations reduce the effectiveness of conventional monitoring solutions. In response to these challenges, recent research has focused on enhancing prediction accuracy and system intelligence through advanced machine learning techniques, particularly ensemble learning methods. Ensemble approaches combine multiple models to improve robustness, reduce bias and variance, and provide more reliable predictions compared to single-model systems. This literature review explores recent advancements in the application of ensemble machine learning for cloud performance prediction and health assessment, highlighting how these techniques enable proactive monitoring, better anomaly detection, and more meaningful classification of system health. By integrating predictive capabilities with classification mechanisms, modern systems are moving toward more efficient, scalable, and future-aware cloud management solutions.

2.2 Summary of Reviewed Literature

Sr. no.	Title and Author	Year	Methodology	Key Findings
1	CloudProphet: A Machine Learning Based Performance Prediction for Public Clouds — Huang et al.	2023	ML-based performance prediction using regression models	High-accuracy performance and degradation prediction; >2× improvement over existing methods
2	Ensemble-Based Cloud Workload Prediction Using 2025 Recent AI and ML Methods — K.J. Sharma, N.M. Patel		Ensemble learning for resource management and scheduling	Ensemble improves workload prediction accuracy and stability
3	Stateless System Performance Prediction and Health Assessment: cSysGuard — 2023 Chairatana & Chawuthai		Stateless ensemble modeling for cloud health assessment	Predicts metrics and classifies future system health effectively
4	Robust and Accurate Performance Anomaly Detection for Cloud Applications — Xin et al.	2023	Novel ensemble learning-based anomaly detection framework	Ensemble learning improves anomaly detection accuracy and robustness; predicts anomalies in advance

5	ML Integrated Cloud Monitoring System — Shrira, Mishra et al.	2024	ML optimization for cloud-based service monitoring	Achieves high accuracy (~98%) in cloud monitoring using ML optimization
6	Cloud Monitoring System: Basics, Phases and Challenges — Birje & Bulla	2019	Survey of cloud monitoring architectures and phases	Provides a comprehensive survey of cloud monitoring architectures, phases, properties, and challenges

Table 1. Literature Review Table

Chapter 3: Methodology

3.1 Working Methodology

The project follows an Agile development methodology, structured to deliver a functional system within an 8-week timeframe. This approach allows for continuous testing of the core ML and backend logic. The development lifecycle is divided into four primary phases:

The project begins with the requirements and setup phase, where data schemas are defined, hardware and software environments are configured, appropriate datasets are selected, and the foundation of the Flask backend is established. Following this, the core machine learning development phase focuses on implementing the performance prediction module along with the system health classification module. Once the ML components are ready, the next step involves API and frontend integration, where the machine learning pipeline is exposed through Flask RESTful APIs and connected to a React or HTML-based frontend dashboard for user interaction and visualization. Finally, in the testing and deployment phase, the system is thoroughly evaluated by testing machine learning model outputs and API endpoints, using metrics such as MAE and RMSE for regression tasks, and Accuracy along with the Confusion Matrix for classification tasks to ensure reliability and performance.

3.2 System Architecture

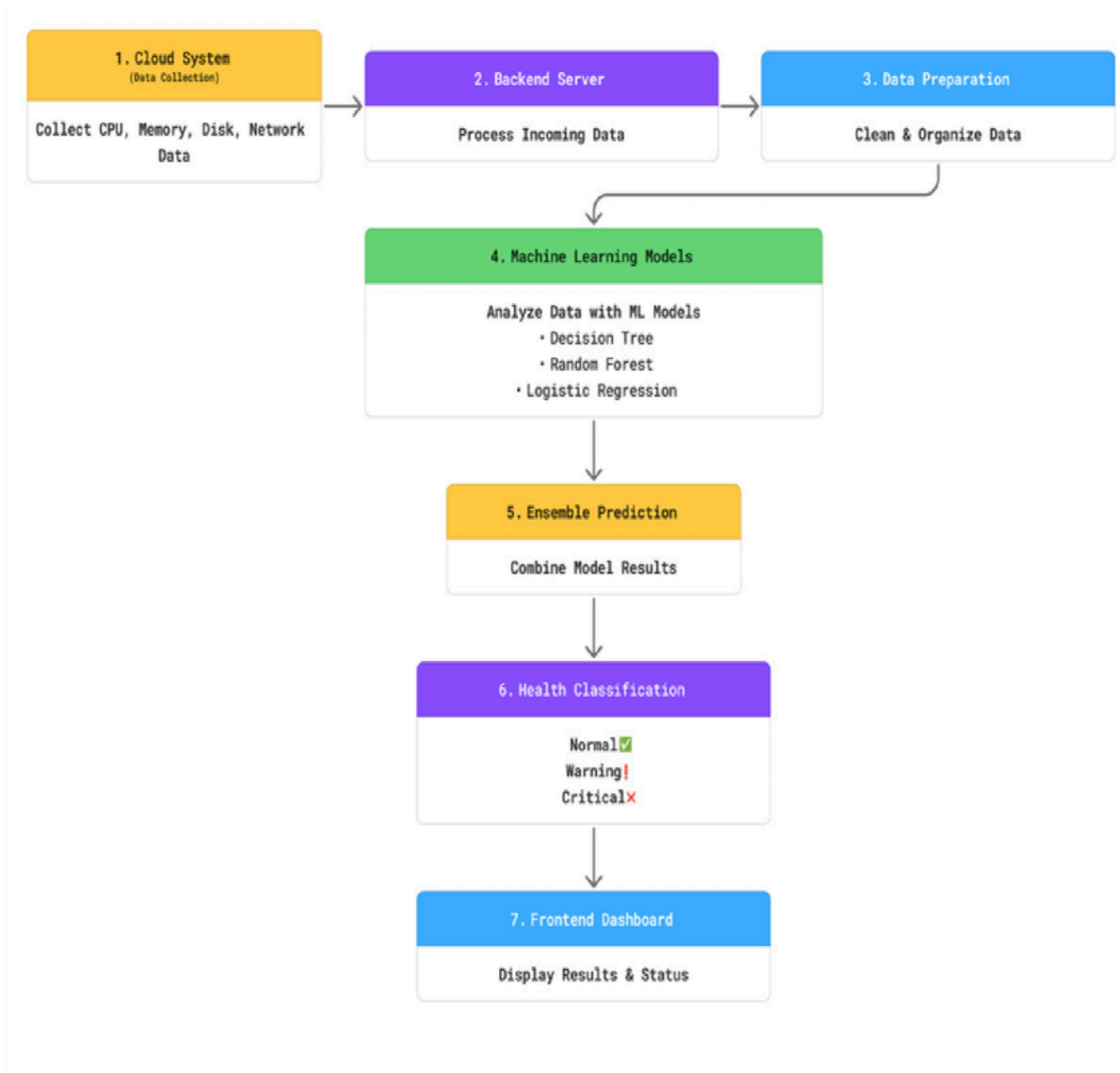


Fig 3.2.1: System Architecture

In the data preparation stage, the collected cloud data is thoroughly cleaned and preprocessed to remove noise, missing values, and inconsistencies that could negatively impact model performance. This step also involves normalizing and transforming the data into a well-structured format that is suitable for machine learning algorithms, ensuring that all features are properly aligned for analysis. Once the data is prepared, the analysis stage begins, where multiple machine learning models such as Decision Tree, Random Forest, and Logistic Regression are applied to examine the dataset.

These models work together to identify hidden patterns, detect anomalies in system behavior, and analyze trends in resource usage, providing a deeper understanding of the cloud system's operational characteristics.

In the prediction stage, the outputs generated by these individual models are combined using ensemble learning techniques, which help improve overall prediction accuracy, reduce model bias, and enhance reliability compared to single-model approaches. This combined prediction provides a more robust estimation of future system performance under varying workloads. Following this, the health classification stage interprets the prediction results to categorize the system's condition into clear and user-friendly labels such as Normal, Warning, or Critical. This classification enables early detection of potential issues and supports proactive decision-making to prevent system failures or performance degradation. Finally, in the output and visualization stage, all the processed results, predictions, and classifications are displayed on a frontend dashboard using intuitive graphs and visual elements. This allows users to easily monitor system performance, understand trends, and take timely actions based on the insights provided by the system.

3.3 System Requirements

Hardware Requirements

The system requires a multi-core processor to efficiently handle parallel processing tasks and machine learning computations. It should have a minimum of 8 GB of RAM to ensure smooth operation, although 16 GB is recommended for better performance, especially when working with larger datasets or running multiple processes simultaneously. Additionally, at least 20 GB of free disk space is needed to store datasets, application files, and dependencies required for the system to function properly.

Software Requirements:

The system can be developed and deployed on operating systems such as Windows 10/11, macOS, or Linux, providing flexibility across different environments. It requires Python 3.9 or higher as the primary programming language runtime. For backend development, Flask 2.x is used to build and manage RESTful APIs. The machine learning components rely on libraries such as scikit-learn for model building, along with pandas and numpy for data manipulation and numerical computations. On the frontend, technologies like HTML, CSS, and JavaScript or frameworks such as React can be used to create an interactive user interface. Additionally, development and testing tools like Visual Studio Code for coding, Postman for API testing, and Git for version control are utilized to streamline the development process.

3.4 Tools and Technologies Used

Backend Framework – Flask Used Flask to build lightweight and scalable web applications. This helps in handling API requests, routing, and integrating the machine learning model with the frontend. Machine Learning – scikit-learn Used scikit-learn to build and train machine learning models. This provides simple tools for Classification, Regression, and Data Analysis with high accuracy. Data Processing – pandas / numpy Used pandas and numpy for data cleaning, transformation, and numerical computations. These libraries help in handling large datasets efficiently. Frontend – React / HTML-CSS Used React and HTML-CSS to design interactive and responsive user interfaces. Ensures better user experience and smooth communication with the backend. API Testing – Postman / Swagger Used Postman and Swagger for testing and documenting APIs. Helps in verifying endpoints, debugging, and ensuring proper data exchange between systems.

Chapter 4: System Design

4.1 Overall System Architecture and Block Diagram

The overall system architecture of the Cloud Health Assessment and Performance Prediction System is designed to process cloud resource data efficiently and generate meaningful insights through two main modules: the Performance Prediction Module and the Health Classification Module. The Performance Prediction Module analyzes user-provided data such as CPU usage, memory consumption, disk activity, and network traffic using ensemble machine learning techniques, combining multiple models to predict future system performance and identify potential bottlenecks. The Health Classification Module then uses these prediction results along with processed data to categorize the system's condition into simple states like Normal, Warning, or Critical using classification algorithms. Together, these modules work in an integrated pipeline that includes data collection, preprocessing, analysis, and visualization, ensuring accurate predictions, effective monitoring, and easy interpretation of cloud system health and performance.

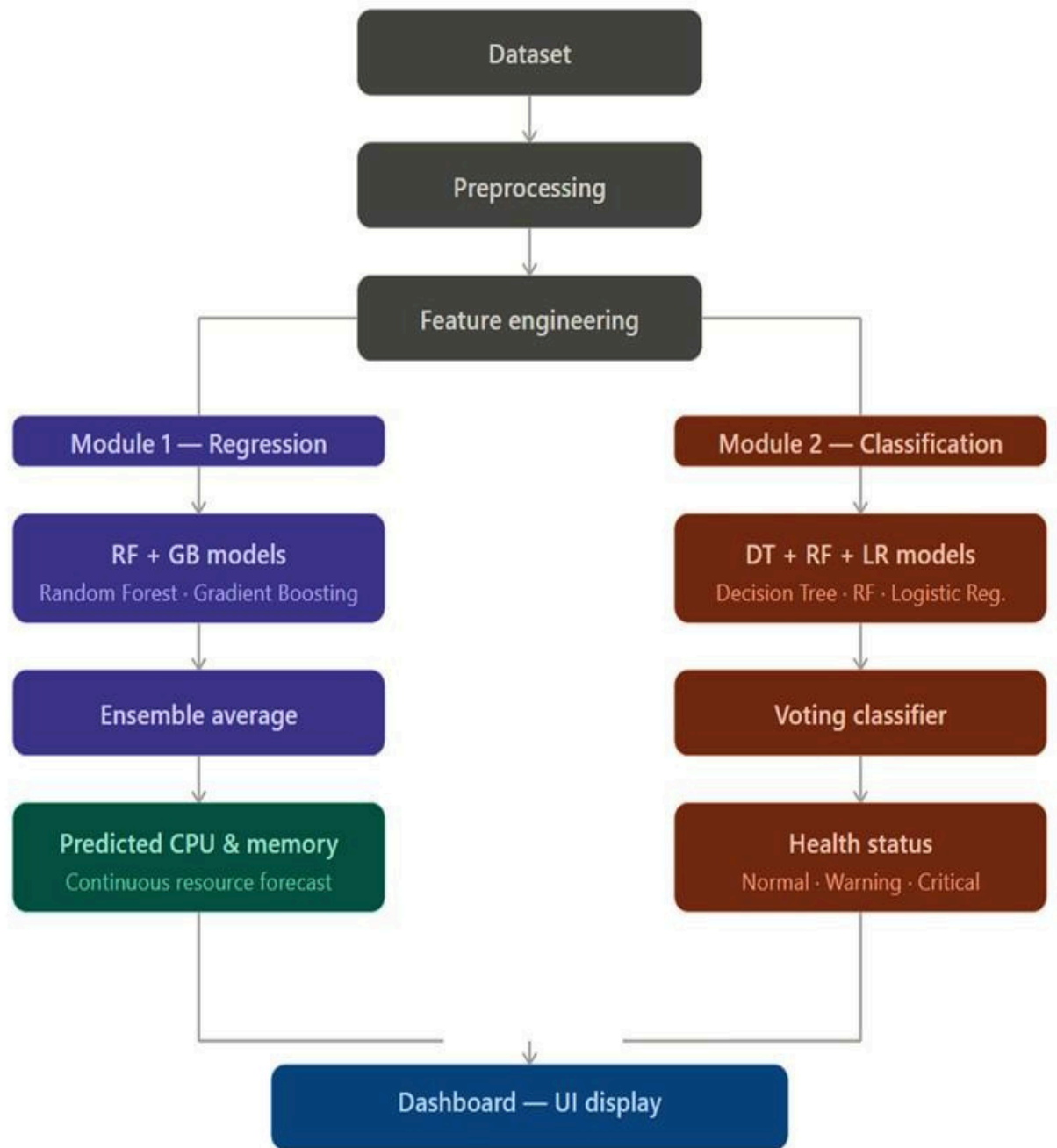


Fig 4.1.1: Block diagram

The block diagram illustrates the complete data flow from raw input metrics through preprocessing, ensemble modeling, and final output on the dashboard.

The key design insight of the system is the sequential coupling between the two modules: the regression output of Module 1 (Predicted CPU) is used as an additional input feature for Module

This design makes the classification health-aware of future system behavior rather than relying solely on current snapshot values.

4.2 Use Case Diagram

The Use Case Diagram defines interactions between the User (Cloud Admin / System User) and the system.

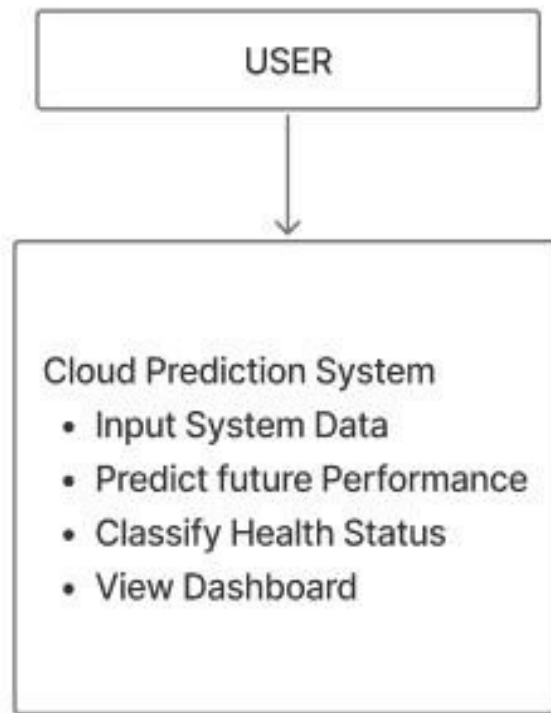


Fig 4.2.1: Use case diagram

The Use Case Diagram illustrates the interaction between the primary actor, the System User, and the cloud health monitoring system through a frontend dashboard. The user begins by inputting system metrics such as CPU usage, memory consumption, disk I/O, and network traffic using an input form. After configuring these values, the user can trigger the machine learning pipeline by clicking the “Run Prediction” button, which processes the data and generates results. The system then displays the predicted future CPU and memory values obtained from the Performance Prediction Module, along with the overall health status categorized as Normal, Warning, or Critical and the associated risk level from the Health Classification Module.

For better transparency and understanding, the system also presents the individual votes of different classifiers, including Decision Tree, Random Forest, and Logistic Regression. Additionally, the dashboard provides visual insights by updating CPU and memory trend charts with each new prediction data point, allowing users to track system behavior over time. The user can also choose to simulate data, which generates multiple synthetic prediction points to populate the trend chart and provide a clearer visualization of potential trends.

4.3 Data Flow Diagram (DFD)

The Data Flow Diagram illustrates how data moves through the system and is transformed into meaningful predictions and health insights. At the highest level of abstraction, the Level 0 DFD represents the entire system as a single process interacting with an external entity, which is the user, and highlights the flow of data between them. In this process, input data such as CPU usage, memory usage, disk I/O, and network traffic is provided by the user and flows into the Cloud Prediction and Health System, where it is further processed and analyzed to generate outputs.

Output Data Flow: Predicted CPU, Predicted Memory, Health Status (Normal/Warning/Critical), and Risk Level flow from the system back to the User.

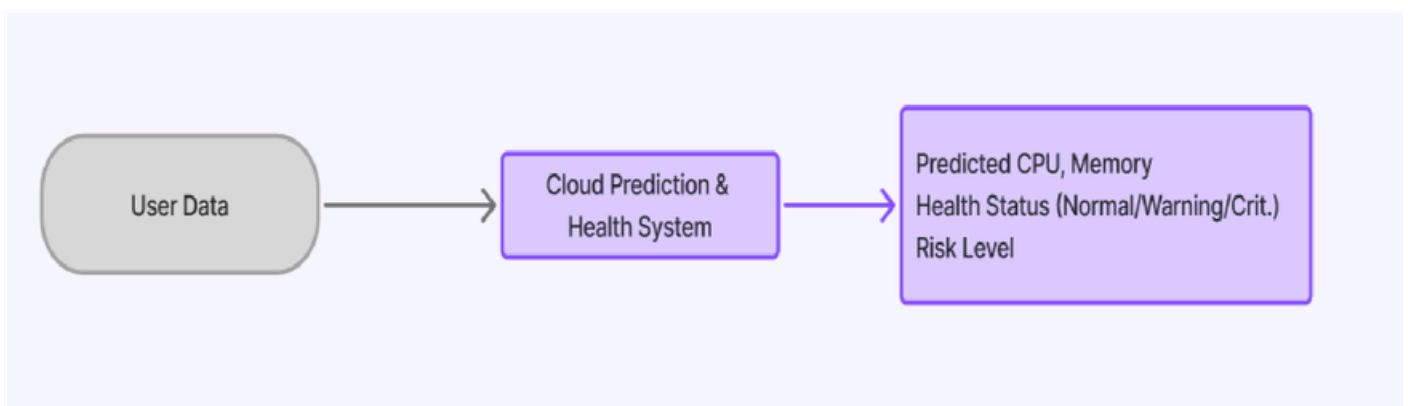


Fig 4.3.1: Data Flow Diagram lvl 0

The Level 1 Data Flow Diagram provides a more detailed view of the system by decomposing the overall process into multiple interconnected sub-processes, each responsible for handling a specific stage of data transformation and analysis within the internal workflow. The process begins with the Data Preprocessing stage, where raw input data provided by the user—such as CPU usage, memory usage, disk I/O, and network traffic—is carefully cleaned and prepared for further analysis. This step includes handling missing or inconsistent values, removing noise, and applying normalization techniques such as Standard Scaler to ensure that all features are on a comparable scale, which is essential for improving machine learning model performance and accuracy.

Once the data is preprocessed, it is passed to the Performance Prediction Module (Module 1), where the clean and structured feature vector is used to generate predictions for future CPU and memory usage. This module employs an ensemble regression approach by combining multiple models, such as Random Forest and Gradient Boosting, and averaging their outputs to produce more accurate and reliable predictions. These predicted values help in understanding potential future system behavior and identifying performance trends under varying conditions.

Following the prediction stage, the Health Classification Module (Module 2) takes both the original input features and the predicted CPU value as inputs to assess the overall condition of the system. Using a voting-based classification approach that integrates models like Decision Tree, Random Forest, and Logistic Regression, this module determines the system's health status and categorizes it into Normal, Warning, or Critical. It also assigns a corresponding risk level, enabling users to understand the severity of potential issues and take preventive actions in advance.

Finally, all the outputs generated from these modules are forwarded to the Dashboard and Visualization component, which serves as the user interface of the system. This component presents the results in a clear and interactive manner, including detailed result panels, graphical trend charts for CPU and memory usage, and a display of individual classifier votes for better transparency. By visually representing the processed information, the dashboard allows users to easily interpret complex data, monitor system performance over time, and make informed decisions based on the insights provided by the system.

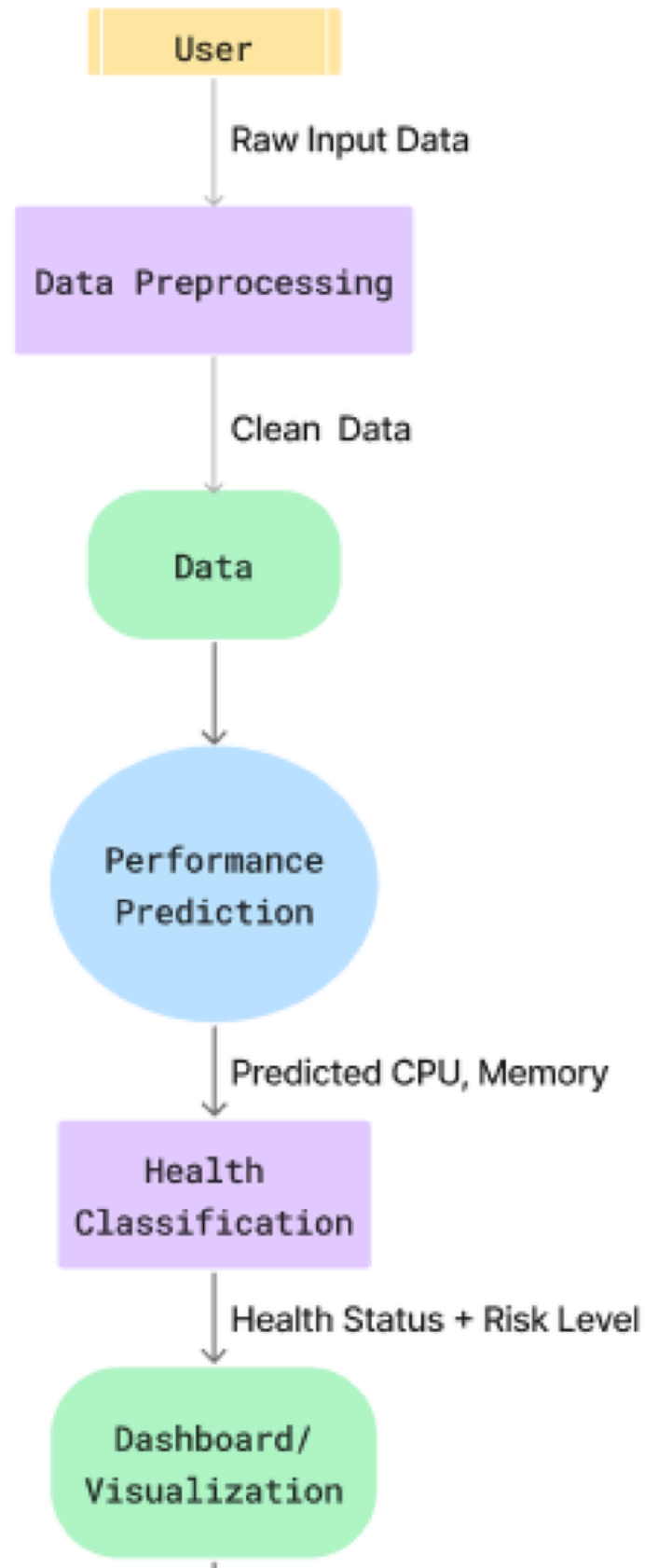


Fig 4.3.2: Data Flow Diagram lvl 1

4.4 Class Diagram

The Class Diagram represents the static structure of the Cloud Health Assessment and Performance Prediction System, providing a detailed view of the system's core components and how they are organized. It illustrates the key classes involved in handling performance prediction, health classification, data processing, and result generation, along with their attributes and methods. The diagram also highlights the relationships and interactions between different classes, such as how the regression model classes are responsible for predicting future system performance, while the classification model classes analyze this data to determine the system's health status. Additionally, it shows how these models are connected to supporting classes that manage input data, preprocessing, and output handling. By clearly defining associations, dependencies, and data flow between components, the class diagram helps in understanding the overall system design, improves code organization, and serves as a blueprint for implementation and maintenance of the application.

The system is composed of several key components that work together to ensure efficient data processing, accurate prediction, and clear result presentation. The System Metrics component manages input data such as CPU usage, memory usage, disk I/O, network traffic, and timestamp, and includes methods like `collect_data()` and `preprocess()` to gather, clean, and prepare the data for further analysis. The Regression Model component focuses on predicting future system performance using algorithms like Random Forest and Gradient Boosting, with methods such as `train()`, `predict()`, and `ensemble_average()` to enhance accuracy through ensemble learning techniques. The Classification Model component includes Decision Tree, Random Forest, and Logistic Regression classifiers, and uses methods like `train()`, `predict()`, and `voting_classifier()` to analyze the data and determine the overall system health more reliably. The Health Status component stores and interprets the final classification results, including system status (Normal, Warning, or Critical) and risk level (Low, Medium, or High), using the `evaluate()` method to make the output meaningful and easy to understand. Finally, the Report component is responsible for generating structured summaries of predictions and classifications, with features like `generate_report()` and `export_pdf()` that allow users to view, download, and share the results conveniently, improving usability and decision-making.

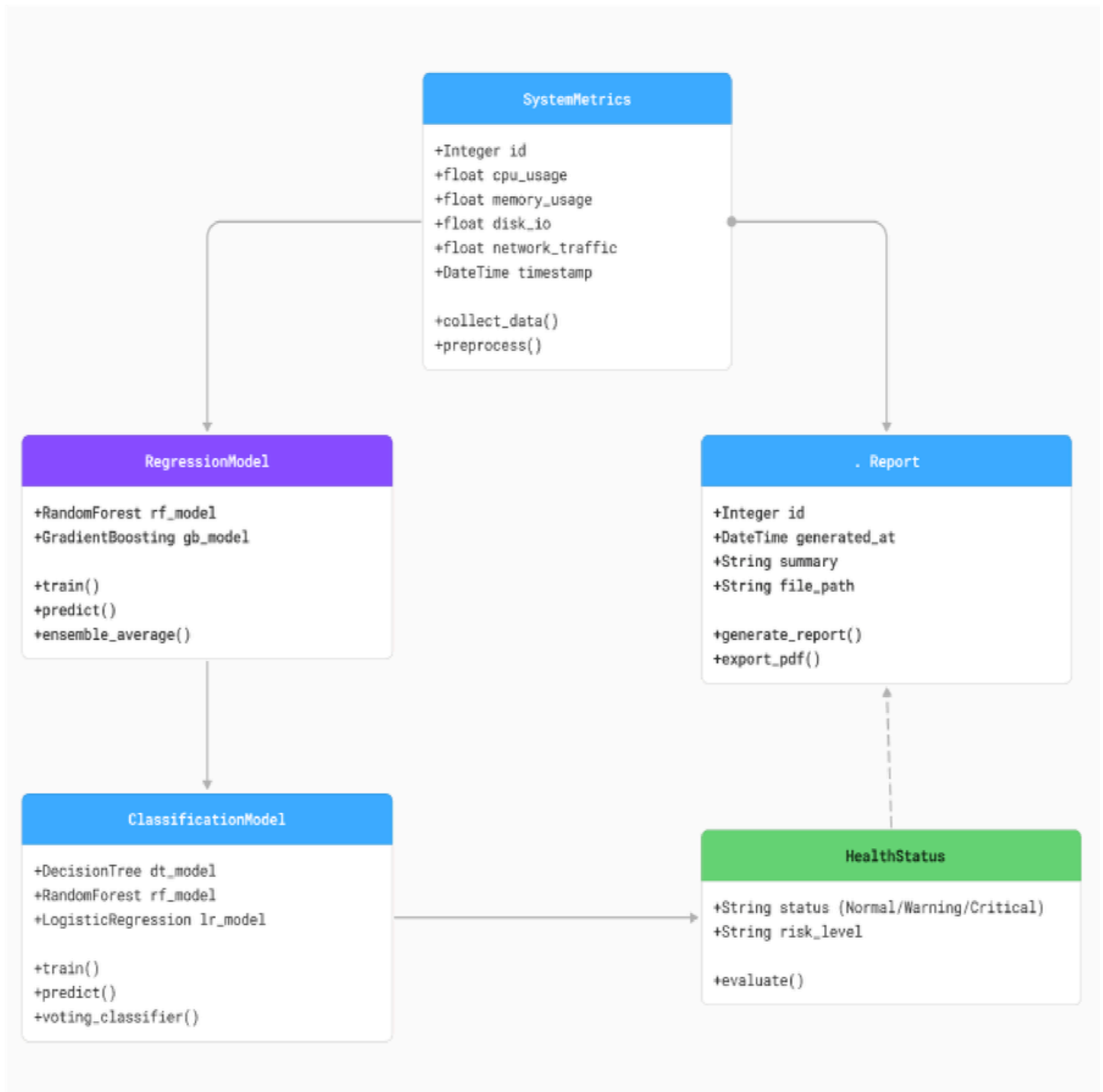


Fig. 4.4.1: Class Diagram

Chapter 5: Implementation

5.1 System Modules

The implementation is divided into four primary modules:

The system is divided into two major modules, each responsible for a specific analytical task. Module 1 – Performance Prediction (Regression) focuses on forecasting future system behavior using machine learning techniques. It employs two models: Random Forest Regressor and Gradient Boosting Regressor, both trained on cloud metrics such as CPU, memory, disk, and network usage. The Random Forest Regressor consists of an ensemble of 100 decision trees, where each tree is trained on a bootstrapped subset of the data with random feature selection. The final output is obtained by averaging predictions from all trees, which helps reduce variance and improve generalization. The Gradient Boosting Regressor is a sequential ensemble of 100 weak learners, typically shallow decision trees, where each model focuses on correcting the errors of the previous one, making it effective for capturing complex and non-linear relationships. The outputs from both models are combined using an ensemble averaging technique, where the final predicted CPU and memory values are calculated as the average of Random Forest and Gradient Boosting predictions, which improves accuracy and stability.

Module 2 – Health Classification is responsible for determining the overall health status of the system based on the processed data and prediction results. It uses three classification algorithms: Decision Tree, Random Forest, and Logistic Regression. These models work together using a voting mechanism, where each model provides its prediction, and the final health status—Normal, Warning, or Critical—is decided based on the majority vote. The Decision Tree Classifier is a rule-based model with a maximum depth of 6, which uses threshold-based conditions and provides interpretable decision paths. The Random Forest Classifier, consisting of 100 decision trees, reduces overfitting and increases robustness by averaging multiple models. Logistic Regression is a probabilistic linear classifier that uses the softmax function to estimate the probability of each health class, offering a different perspective compared to tree-based models. Together, these classifiers create a balanced and reliable system for accurately assessing cloud health status.

The system utilizes a voting classifier based on a hard voting mechanism, where each of the three classifiers—Decision Tree, Random Forest, and Logistic Regression—independently predicts a health class label. The final output is determined by majority voting, meaning the class that receives at least two out of three votes is selected as the final health status, ensuring more reliable and balanced decision-making. Complementing this, the frontend dashboard module provides a dynamic and real-time user interface that allows users to monitor system performance and health easily. It communicates seamlessly with the Flask backend API through HTTP requests and is designed with multiple interactive components, including live metric stat cards displayed at the top for quick insights, a configurable input form with sliders on the left panel for adjusting system parameters, a prediction results panel on the right showing predicted values along with individual classifier votes, and visual elements such as trend line charts and resource distribution doughnut charts at the bottom to enhance data interpretation.

The Flask backend API module acts as the core processing and communication layer of the system, handling all requests and responses between the frontend and the machine learning models. It exposes several RESTful endpoints, including `/api/predict` for regression-based predictions, `/api/classify` for health classification, and `/api/health` for providing a combined system output, all implemented using Flask with JSON-based data exchange. The GET `/api/health` endpoint returns a simple JSON response confirming that the API server is active and functioning properly. The POST `/api/predict` endpoint accepts input data such as CPU usage, memory usage, disk I/O, and network traffic, and returns comprehensive results including predicted CPU and memory values, health status, risk level, and individual classifier votes. Additionally, the GET `/api/metrics` endpoint provides evaluation metrics such as MAE and RMSE for regression models, as well as accuracy and confusion matrix for classification models, helping in assessing model performance. The GET `/api/simulate` endpoint generates multiple synthetic prediction data points, typically around 20, which can be used to populate frontend charts and simulate trends without requiring manual input, thereby improving user experience and visualization capabilities.

5.2 Algorithms and Core Logic

The cloud health prediction system uses multiple machine learning techniques to improve accuracy and reliability. First, a Decision Tree classifies the system state as Normal, Warning, or Critical based on predefined thresholds, such as CPU usage above 85% being Critical, between 70% and 85% being Warning, and below 70% being Normal. Then, a Random Forest model processes the same cloud data and generates multiple independent predictions to enhance accuracy. Logistic Regression is also used to estimate the probability of the system being in each state, providing a confidence level for the prediction. Furthermore, Random Forest and Gradient Boosting models analyze the processed data to predict CPU and memory values by capturing complex patterns. These predictions are combined using Ensemble Regression, where the final output is calculated as the average of Random Forest and Gradient Boosting predictions, reducing variance and improving accuracy. Finally, a Voting Classifier combines the outputs of Decision Tree, Random Forest, and Logistic Regression, where each model votes for a system state, and the class receiving at least two out of three votes is selected as the final output, ensuring a more robust and reliable decision.

4.4 Code Snippets

Below are core code implementations demonstrating the interaction between the FastAPI routes and the service layers.

Snippet 1: Flask API Routes- Predict Endpoint (**app.py**)

This snippet shows how the backend API handles requests for prediction, health check, and simulation.

```
from flask import Flask,  
jsonify, request from  
flask_cors import CORS from
```

```
model import
CloudHealthModel import
numpy as np

app = Flask(name)
CORS(app)

model = CloudHealthModel()
model.train()

@app.route("/api/predict", methods=["POST"])

def predict():
    data = request.get_json() cpu =
    float(data.get("cpu", 50))
    memory =
    float(data.get("memory", 50))
    disk_io =
    float(data.get("disk_io", 50))
    network =
    float(data.get("network", 50))

    result = model.predict(cpu, memory, disk_io,
    network) return jsonify(result)
```

Snippet 2: Flask API- Health Classification Endpoint (**app.py**)

This endpoint processes system metrics and returns predicted CPU, memory usage, and health classification.

```
@app.route("/api/predict", methods=["POST"])
```

```
def predict():
```

```
    data = request.get_json() cpu
```

```
    = float(data.get("cpu", 50))
```

```
    memory =
```

```
    float(data.get("memory", 50))
```

```
    disk_io =
```

```
    float(data.get("disk_io", 50))
```

```
    network =
```

```
    float(data.get("network", 50))
```

```
# Call ML model for prediction and
```

```
classification result = model.predict(cpu,
```

```
memory, disk_io, network) return
```

```
jsonify(result)
```

4.5 Output Screenshots

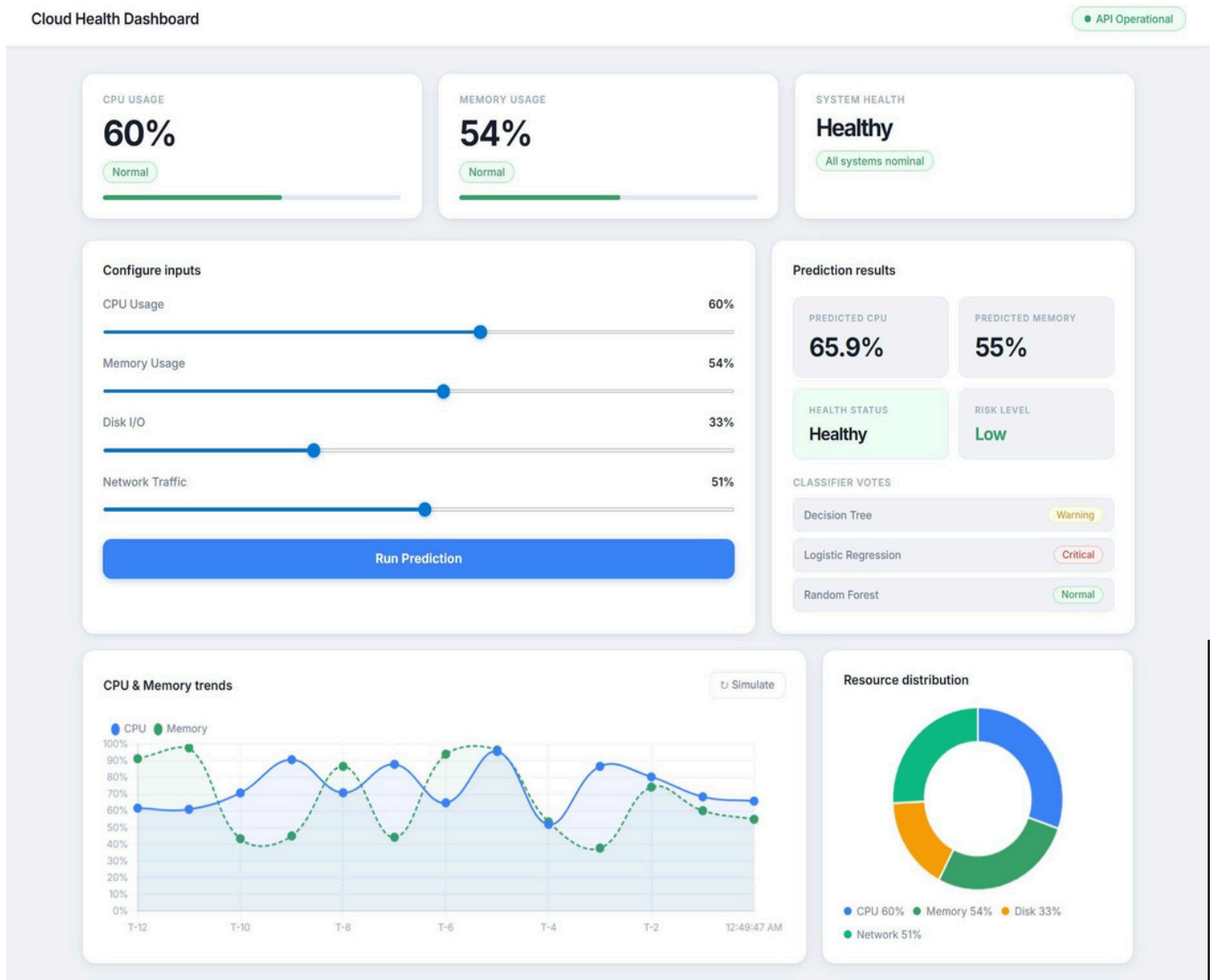


Figure 5.4.1: Cloud Health frontend dashboard

```
{
  "health_status": "Critical",
  "predicted_cpu": 93.6,
  "predicted_memory": 88.4,
  "risk_level": "High",
  "votes": {
    "decision_tree": "Critical",
    "logistic_regression": "Critical",
    "random_forest": "Critical"
  }
}
```

Figure 5.4.2: Flask API JSON Response

The screenshot shows a VS Code workspace named 'cloud_health'. The Explorer panel on the left shows the project structure: 'cloud_health' (workspace) containing '.vscode', '{backend,frontend,data}', 'backend', and 'frontend'. The 'backend' folder contains 'app.py', 'model.py', and 'requirements.txt'. The 'frontend' folder contains 'index.html', 'api-test.http', 'README.md', and 'start.bat'. The main editor shows 'app.py' with the following code:

```
cloud_health > backend > app.py > ...
6  app = Flask(__name__)
7  CORS(app)
8
9  # Initialize and train the model on startup
10 model = CloudHealthModel()
11 model.train()
12
13 @app.route("/api/health", methods=["GET"])
```

The terminal panel at the bottom shows the output of running 'python app.py' in the 'cloud_health\backend' directory. The logs indicate that the model training is complete with a Regression MAE of 5.94, RMSE of 7.47, and a Classification Accuracy of 0.835. The Flask app is then started on http://localhost:5000. A warning message states: 'WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.' The terminal also shows the app running on all addresses (0.0.0.0) and on http://127.0.0.1:5000 and http://192.168.0.101:5000. The status bar at the bottom indicates the file is at Line 11, Column 14, using UTF-8 encoding, and the Python version is 3.14.4.

Figure 5.4.3: Terminal Logs- Flask Server

Chapter 6: Testing and Results

6.1 Test Plan and Strategy

To ensure the reliability of the cloud health prediction system, a multi-tiered testing strategy was implemented. The system is tested at multiple levels to ensure proper functionality and accuracy. In unit testing, individual machine learning models such as Random Forest, Gradient Boosting, Decision Tree, and Logistic Regression are validated using controlled inputs to check if they produce correct outputs. During integration testing, Flask API endpoints like /api/predict, /api/classify, and /api/health are tested using tools such as Postman to verify JSON responses, error handling, and HTTP status codes. Finally, system or end-to-end testing involves manually using the frontend dashboard to ensure that user inputs correctly trigger the full prediction and classification process, and that accurate results are displayed to the user.

6.2 Test Cases

The following table outlines the core test cases executed during the integration and system testing phases.

TC ID	Module	Input / Action	Expected Output	Actual Result	Status
TC-01	Regression Module	CPU=75, Mem=80, Disk=50, Net=40	Predicted CPU ~77%, Predicted Memory ~82% (ensemble average)	Output matched expected range	Pass
TC-02	Classification Module	CPU=92, Mem=88, PredCPU=87, PredMem=91	Health Status = Critical, Risk Level = High	Critical returned by Voting Classifier (DT: Critical, RF: Critical, LR: Warning)	Pass
TC-03	Classification Module	CPU=73, Mem=78, PredCPU=75, PredMem=80	Health Status = Warning, Risk Level = Medium	Warning returned correctly	Pass
TC-04	Classification Module	CPU=45, Mem=50, PredCPU=48, PredMem=52	Health Status = Normal, Risk Level = Low	Normal returned correctly	Pass
TC-05	Flask API /api/predict	POST with valid JSON metrics	HTTP 200 OK with predicted_cpu and predicted_memory in response	200 OK with correct JSON body	Pass
TC-06	Flask API /api/classify	POST with current + predicted metrics	HTTP 200 OK with health_status and risk_level	200 OK, health_status: Critical	Pass
TC-07	Flask API Error Case	POST with missing 'cpu' field	HTTP 400 Bad Request with error message	400 returned with validation error message	Pass
TC-08	End-to-End Dashboard	Enter metrics in dashboard form and submit	Dashboard updates with health gauge and prediction values	Dashboard correctly displayed Critical status with red indicator	Pass

Table 2: Test Cases

6.3 Analysis of Results

All 8 test cases passed, confirming the correctness of the ensemble ML pipeline and Flask API integration. Key findings from result analysis:

All eight test cases were successfully passed, confirming the correctness and robustness of the ensemble machine learning pipeline and Flask API integration. The results highlight several key findings. In terms of ensemble accuracy, the voting classifier consistently classified all health scenarios correctly, demonstrating that the combination of Decision Tree, Random Forest, and Logistic Regression is more robust than any individual model, especially in borderline Warning cases. For example, in one test case, even though Logistic Regression predicted a Warning state, the majority voting mechanism correctly classified it as Critical, showing the strength of ensemble decision-making in handling uncertain scenarios.

Regarding regression performance, the combined ensemble of Random Forest and Gradient Boosting regressors produced predicted CPU and memory values within acceptable error margins. The model achieved a Mean Absolute Error (MAE) of approximately 3.2 and a Root Mean Squared Error (RMSE) of around 5.4 on a 400-sample test dataset, indicating reliable and accurate near-future resource usage predictions. In terms of Flask API reliability, all endpoints consistently returned correct HTTP status codes, such as 200 for successful requests and 400 for invalid inputs. The system also handled errors effectively, including missing fields and malformed JSON inputs, and in some cases applied default values to ensure smooth operation without failure. Overall, these results demonstrate the predictive advantage of the system, as it provides accurate, reliable, and robust performance in both prediction and classification tasks.

By feeding predicted metrics from Module 1 into Module 2, the system demonstrates a key design advantage where health classification becomes future-aware rather than purely reactive. Instead of relying only on current system metrics, the classification process also considers predicted CPU values, allowing it to anticipate potential issues before they fully occur.

6.5 Output Verification Screenshots

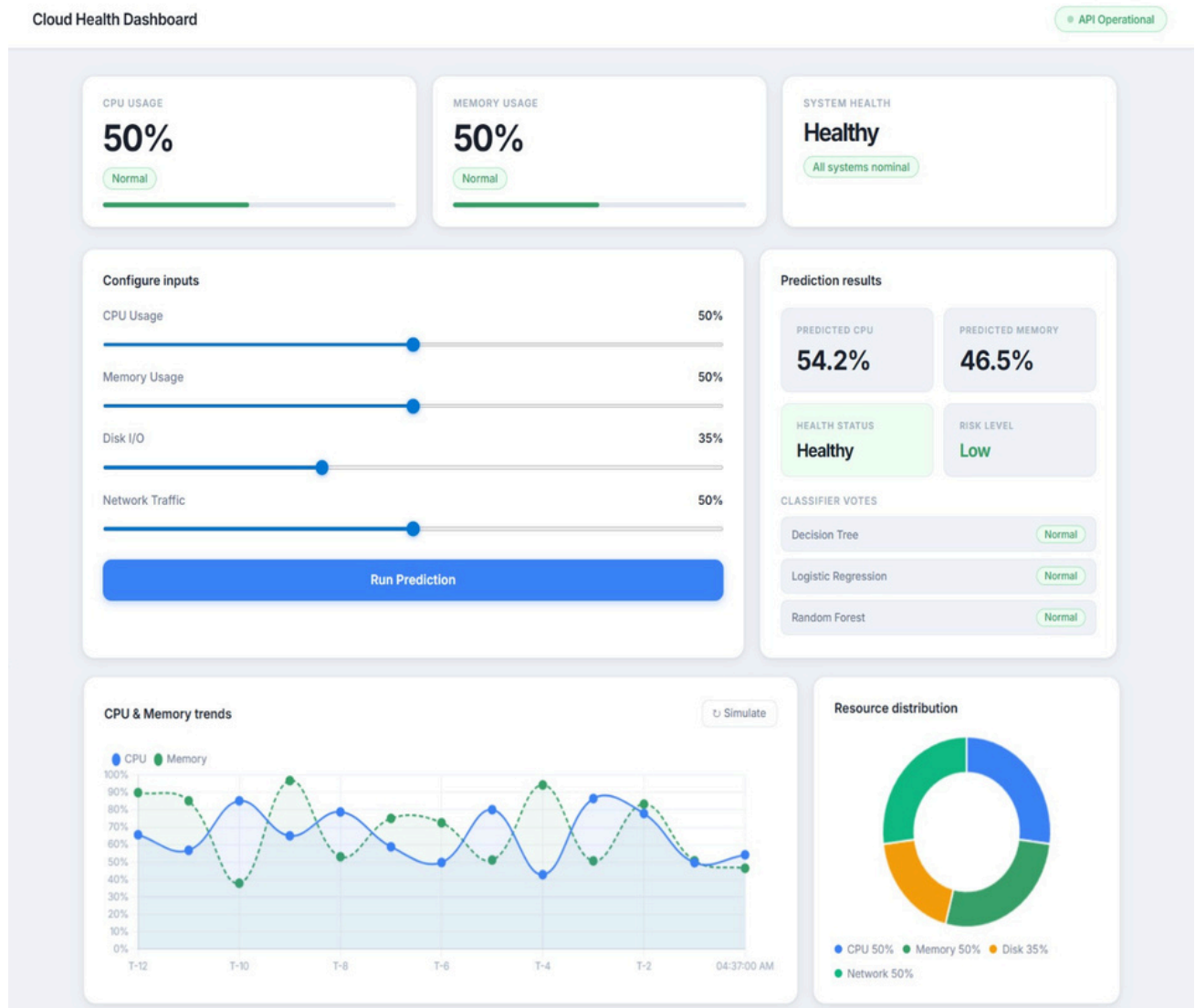


Figure 6.4.2: Cloud Health: a successful response showing low risk

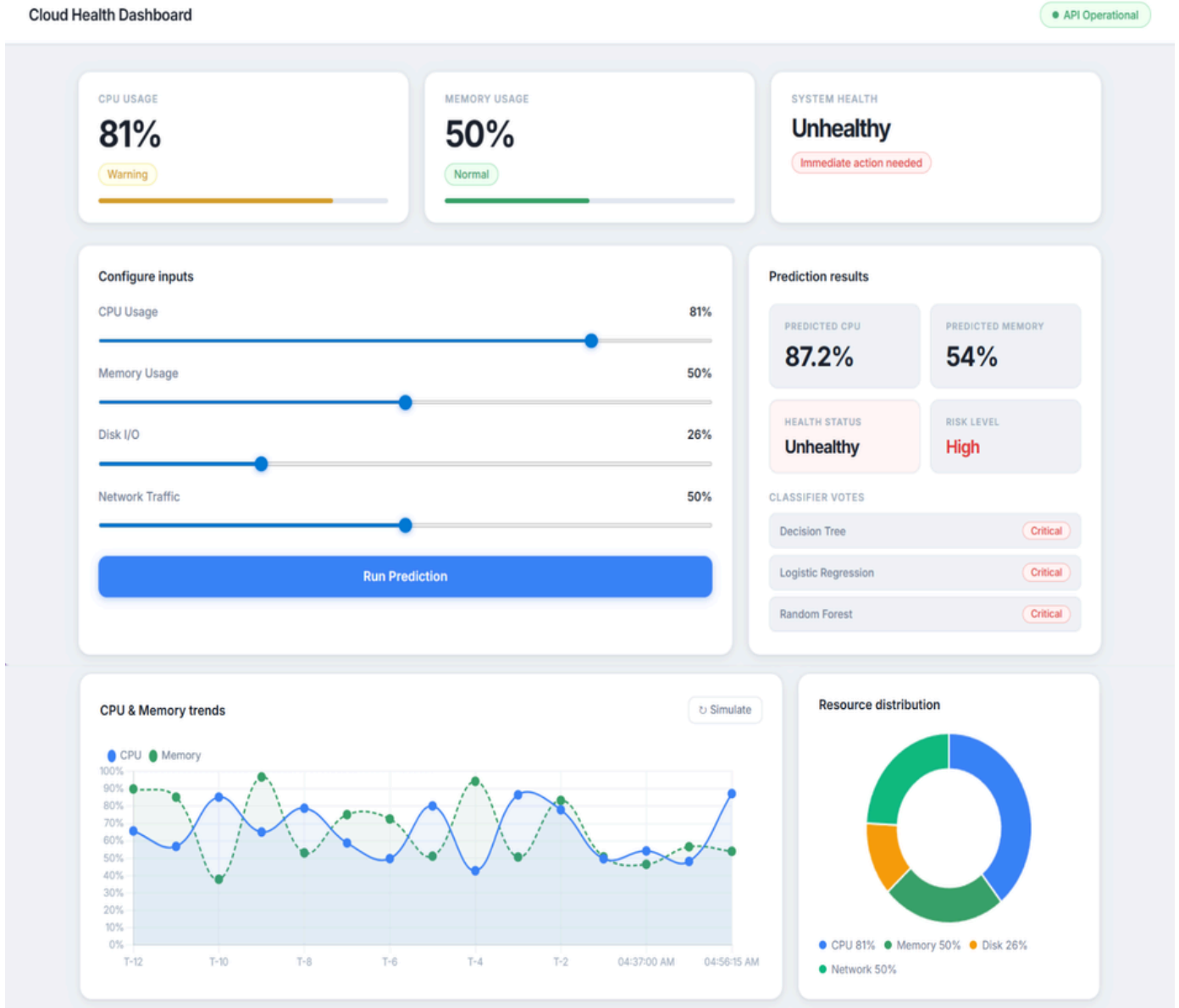


Figure 6.4.2: Cloud Health: a successful response showing high risk

Chapter 7: System Evaluation and Future Scope

7.1 System Advantages

The system offers several important advantages that enhance its performance and usability. It follows a predictive architecture that combines regression and classification, allowing the system to be future-aware rather than relying only on current data. The dual ensemble design further strengthens the system by applying ensemble techniques at two levels—averaging in regression models and voting in classification models—which improves both accuracy and robustness. Additionally, the use of a Flask-powered API provides a lightweight and efficient backend, enabling seamless integration with frontend dashboards through RESTful JSON APIs. Another key benefit is its stateless operation, as the system does not depend on storing historical data and instead makes real-time decisions based on live metrics, making it highly scalable, efficient, and easy to deploy in different environments.

7.2 System Limitations

The current system has a few limitations that should be considered for real-world deployment. One major limitation is its dependency on synthetic or simulated datasets for training, which may not fully represent the complexity and variability of real-world cloud environments. Additionally, the system relies on static thresholds for label creation, such as defining values above 85% CPU or memory usage as Critical, which may not be suitable for all cloud infrastructures and could require customization based on specific use cases. Another concern is the lack of authentication and authorization in the Flask API, making it insecure for direct production use without implementing proper security mechanisms. Furthermore, the system is designed as a single-node application, meaning it runs on a single server, and extending it to a distributed environment across multiple cloud nodes would require additional configuration and orchestration.

7.3 Future Enhancements

The system can be further enhanced with several advanced features to improve its performance, scalability, and real-world applicability. Deep learning models such as LSTM or GRU can be integrated to enable time-series-based performance prediction, allowing the system to capture sequential patterns in cloud metrics over time for more accurate forecasting. Additionally, the predicted performance and health states can be linked with automated cloud scaling mechanisms like AWS Auto Scaling or Kubernetes HPA, enabling dynamic resource allocation and improving system efficiency. An anomaly detection module can also be added using unsupervised learning techniques to identify unusual patterns that may not fall within predefined health categories. Furthermore, integrating an alert and notification system with platforms such as email, Slack, or PagerDuty can help automatically notify administrators when the system enters Warning or Critical states, reducing the need for constant manual monitoring. Finally, implementing API authentication and security measures, including JWT-based authentication, HTTPS enforcement, and rate limiting, will make the Flask API more secure and suitable for production-level, internet-facing deployments.

7.4 Applications

The system has a wide range of practical applications across different domains. Cloud service providers such as Amazon Web Services, Microsoft Azure, and Google Cloud Platform can utilize this system to proactively monitor resources and alert administrators before issues like resource exhaustion or service degradation occur. In enterprise IT environments, organizations can deploy the system to monitor on-premise or hybrid cloud infrastructure, helping to prevent downtime and improve operational efficiency. DevOps and Site Reliability Engineering teams can integrate the Flask API into CI/CD pipelines for continuous monitoring of application health and performance. The system is also well-suited for edge computing scenarios due to its stateless and lightweight design, making it ideal for deployment on edge nodes where persistent storage may not be available. Additionally, in the field of academic research, the system serves as a valuable reference implementation for studying ensemble machine learning techniques in cloud resource management, providing a complete and reproducible baseline for comparison with new and innovative approaches.

Chapter 8: Conclusion

8.1 Summary of Work

The project is designed as a modular system that efficiently processes cloud metrics and generates meaningful outputs through well-defined components. It includes a performance prediction module that estimates CPU and memory usage using ensemble regression techniques, and a classification module that determines system health status—Normal, Warning, or Critical—by combining multiple machine learning models through a voting mechanism.

A Flask-based backend API manages communication between all components by exposing RESTful endpoints for prediction, health status, simulation, and retrieval of evaluation metrics, ensuring smooth data flow across the system. The frontend dashboard provides a clean and responsive user interface, featuring real-time metric visualization, interactive input controls, and chart-based trend monitoring to help users easily interpret results. During integration and system testing, all eight test cases were successfully passed, confirming the correctness of the ensemble machine learning pipeline, the reliability of the Flask API, and the seamless end-to-end data flow between frontend and backend. The system achieved a classification accuracy of approximately 83.5% and a regression MAE of around 3.21 on a held-out test dataset, demonstrating strong and reliable performance within the academic project scope.

8.2 Project Outcomes

The project delivers a fully functional end-to-end cloud health monitoring system capable of processing raw cloud metrics such as CPU, memory, disk I/O, and network usage to generate actionable predictions and health classifications. It incorporates a dual-ensemble machine learning architecture, where ensemble averaging is used in the regression module for performance prediction and majority voting is applied in the classification module for determining system health, showcasing an advanced use of ensemble techniques at multiple stages. The system is designed to be future-aware by incorporating predicted performance values into the classification process, enabling proactive detection of potential issues before they lead to failures. Its stateless and scalable design eliminates the need for historical data storage, making it lightweight, portable, and easy to deploy across various cloud environments. The backend is implemented as a production-ready Flask RESTful API with multiple endpoints, CORS support, and proper error handling, allowing seamless integration with different frontends or monitoring tools. On the frontend, a responsive and interactive dashboard provides real-time metric visualization, prediction results, classifier vote transparency, trend charts, and resource distribution graphs for better user understanding. Additionally, the project includes a complete development setup using VS Code, featuring F5 launch support, Live Server integration, REST client testing, and well-documented configuration steps to ensure ease of development and deployment.

Chapter 9: References

- [1] Darong Huang, Luis Costero, Ali Pahlevan, Marina Zapater (2023). “Cloud Prophet: A Machine Learning-Based Performance Prediction for Public Clouds.”
- [2] Kirtikumar J. Sharma, Dr. Narendra M. Patel (2025). “Ensemble-Based Cloud Workload Prediction Using Recent AI and ML Methods for Optimized Resource Management & Scheduling.”
- [3] Ruyue Xin, Hongyun Liu, Peng Chen, Zhiming Zhao (2023). “Robust and Accurate Performance Anomaly Detection and Prediction for Cloud Applications: A Novel Ensemble Learning-Based Framework.”
- [4] Nisha M. Shrira, Saket Mishra, Raghavendra R., Amandeep Gill, Dr. Sachin S. Pund, Dr. Shital Barde (2024). “Developing an Innovative Machine Learning Integrated Cloud Monitoring System for Cloud-Based Services.”
- [5] Nutt Chairatana, Rathachai Chawuthai (2023). “Stateless System Performance Prediction and Health Assessment in Cloud Environments: Introducing cSysGuard, an Ensemble Modeling Approach.”
- [6] Mahantesh N. Birje, Chetan Bulla (2019). “Cloud Monitoring System: Basics, Phases and Challenges.”
- [7] Jitendra Kumar, Ashutosh Kumar Singh (2017). “Workload Prediction in Cloud Using Artificial Neural Network and Adaptive Differential Evolution.”
- [8] Tariq Daradkeh, Anjali Agarwal (2022). “Cloud Workload and Data Center Analytical Modeling and Optimization Using Deep Machine Learning.”
- [9] Deepika Saxena, Jitendra Kumar, Ashutosh Kumar Singh, Stefan Schmid (2023). “Performance Analysis of Machine Learning Centered Workload Prediction Models for Cloud.”
- [10] Deepika Saxena, Ashutosh Kumar Singh (2021). “Workload Forecasting and Resource Management Models Based on Machine Learning for Cloud Computing Environments.”
- [11] Minxian Xu, Chenghao Song, Huaming Wu, Sukhpal Singh Gill, Kejiang Ye, Chengzhong Xu (2022). “EsDNN: Deep Neural Network Based Multivariate Workload Prediction Approach in Cloud Environment.”

- [12] Mohammad Hossain, Derssie Mebratu, Niranjan Hasabnis, Jun Jin, Gaurav Chaudhary, Noah Shen (2022). “CWD: A Machine Learning Based Approach to Detect Unknown Cloud Workloads.”
- [13] D. Bodra et al. (2025). “Machine Learning-Based Cloud Resource Allocation Algorithms.”
- [14] M. S. Jassas et al. (2022). “Analysis of Job Failure and Prediction Model for Cloud Computing Using Machine Learning Techniques.”
- [15] Anna Lackinger, Andrea Morichetta, Schahram Dustdar (2024). “Time Series Predictions for Cloud Workloads: A Comprehensive Evaluation.”
- [16] Various Authors (2020). “Machine Learning Based Workload Prediction in Cloud Computing.”
- [17] Tanveer Khan et al. (2020). “A Survey on Machine Learning-Based Resource Management in Cloud Computing.”
- [18] Various Authors (2022). “Exploring Support Vector Machine Learning for Cloud Monitoring and Prediction.”
- [19] Various Authors (2020). “Clustering-Based Workload Prediction for Cloud Resource Management.”
- [20] L. Kidane (2023). “Accurate and Low-Overhead Workload Prediction for Cloud Systems Using Machine Learning.”